



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Real-Time UAV Detection and Tracking on Mobile Platform Using CNN

MLOps Tutorial Report

submitted by

Sachit Ramesha Gowda	1RV23AI084
Ravi Kishan Kumar	1RV22AI080
Rayala Yuvaraj Vaishnav	1RV23AI081
S Vishwanatha	1RV23AI083

Artificial Intelligence and Machine Learning

2024-2025

TABLE OF CONTENTS		
Phase 1:Model Development and Foundations		
Chapter 1		
1.1	Problem Statement and Objectives	4
1.2	Project Scope and Evaluation Metrics	5
1.3	Tools and Environmental Setup	6-7
1.4	Literature Review	8-10
Chapter 2		
2.1	Understanding MLOps	11
2.2	Core MLOps Principles	11
2.3	Requirement Analysis	12
2.4	Risk Matrix	13
2.5	Risk Analysis	14
Chapter 3		
3.1	Data Collection and Exploration Techniques	15
3.2	Data Profiling, cleaning and preprocessing	15
3.3	Feature Extraction and Feature Selection	16
3.4	Model Design Approaches	16
3.5	Governance and Data Version Control	16-17
Chapter 4		
4.1	Model Building	18
4.2	Training, Validation and Testing	18
4.3	Evaluation Metrics and Visualisation	18-19
4.4	Experiment Tracking	19-22
Phase 2:Deployment, Monitoring and Governance		
Chapter 5		
5.1	Revisiting Model Performance	23
5.2	Hyperparameter Tuning and Performance	23
5.3	Interpretability	24
5.4	Model Versioning and Registry	24
5.5	Documentation of Model Updates	24
Chapter 6		

6.1	Overview of Deployment Workflow for Offline Edge ML	25
6.2	Tools Setup for Reproducible Training and Deployment	25-26
6.3	Automating Model Training and Testing (Offline):	26
Chapter 7		
7.1	Model Monitoring Concepts and KPI's	27-28
7.2	Detecting Concept Drift and Data Drift	28
7.3	Model Retraining and Redeployment	29-30
7.4	Logging,Alerting and Dashboard Setup	30-31
Chapter 8		
8.1	Post-Deployment Performance Review	32
8.2	Continuous Improvement via Feedback Loops	32-33
8.3	Governance and Ethical AI Practices	33
8.4	Documentation: Model Cards, Data Cards	33-34
8.5	Audit and Review Checklist	35
Appendix		
A.1	List Of MLOps Tools, Its Features, Pros And Cons	36-37
A.2	Tool Installation Guides	37
A.3	References	37-38
A.4	Folder Structure	39
A.5	Further Reading	39

Chapter 1

Introduction

1.1 Problem Statement and Objectives

1.1.1. Problem Statement

Unmanned Aerial Vehicles (UAVs), commonly known as drones, are increasingly deployed in both civilian and defense applications such as surveillance, traffic monitoring, disaster management, and security enforcement. With the rapid growth of UAV usage, real-time detection and tracking of drones have become critical to ensure safety, situational awareness, and threat mitigation.

Traditional detection systems struggle with dynamic environments, varying UAV sizes, lighting conditions, and background clutter. Recent advances in AI-based computer vision, particularly **Convolutional Neural Networks (CNNs)**, enable automated UAV detection from images and video streams. However, most existing UAV datasets are single-class and lack environmental diversity, which limits model generalization and robustness in real-world scenarios.

Additionally, deploying such models on mobile or edge platforms introduces constraints related to latency, computational efficiency, and continuous model reliability. Therefore, accurate bounding box detection, real-time inference on mobile platforms, and automated machine learning lifecycle management using MLOps practices are essential to build a scalable and reliable UAV detection system.

1.1.2. Objectives

The primary objective of this project is to design and deploy a robust real-time UAV detection and tracking system using a YOLO-based Convolutional Neural Network (CNN). The system aims to accurately detect UAVs with precise bounding boxes, track their movement across video frames, and perform efficient inference on mobile or edge platforms. To ensure scalability, reliability, and continuous improvement, the project integrates an MLOps pipeline that manages the entire machine learning lifecycle from experimentation to deployment and monitoring.

The specific objectives of this project are:

- Perform precise bounding box detection and enable real-time UAV tracking across video streams

- Optimize the model for real-time inference on mobile or edge devices with limited computational resources
- Implement a complete MLOps pipeline that includes:
 - Model and data versioning for reproducibility
 - Experiment tracking to evaluate and compare model performance
 - Automated model deployment to production environments
 - Performance monitoring and lifecycle management to handle model drift and updates

1.2 Project Scope and Evaluation Metrics

1.2.1. Project Scope

This project focuses on developing a real-time UAV (drone) detection and tracking system using a YOLO-based Convolutional Neural Network (CNN) integrated with MLOps practices. The scope includes designing, training, and evaluating an object detection model capable of accurately identifying UAVs with bounding boxes and tracking their movement in video streams under diverse environmental conditions.

To address real-world deployment challenges, the system is optimized for offline execution in environments with limited internet connectivity and constrained computational resources, such as mobile or edge devices. Data augmentation techniques are applied to improve model generalization across varying backgrounds, lighting conditions, and UAV scales.

The project incorporates an end-to-end MLOps pipeline to manage the machine learning lifecycle, including dataset and model versioning, experiment tracking, and reproducible training workflows. A lightweight offline dashboard is developed to simulate real-time UAV detection, visualization, and monitoring without reliance on cloud services, enabling deployment in resource-constrained and disconnected environments.

1.2.2. Evaluation Metrics

To evaluate the performance of the UAV detection model, standard object detection metrics are used, namely mAP@50, Precision, and Recall. These metrics collectively measure detection accuracy, localization quality, and robustness in real-time scenarios.

Mean Average Precision (mAP@50)

Mean Average Precision at IoU threshold 0.5 (mAP@50) is the primary evaluation metric used in this project. It measures how accurately the model detects UAVs with correctly localized bounding boxes. A detection is considered correct if the Intersection over Union (IoU) between the predicted bounding box and the ground truth exceeds 0.5. Higher mAP@50 values indicate better overall detection performance and localization accuracy.

Precision

Precision measures the proportion of correctly detected UAVs out of all UAV detections made by the model. It reflects the model's ability to minimize false positives, which is critical in surveillance and security applications where incorrect detections can trigger false alarms.

Recall

Recall measures the proportion of actual UAVs that are successfully detected by the model. It indicates the model's effectiveness in minimizing false negatives, ensuring that UAVs present in the scene are not missed during detection.

Metric Relevance

- **mAP@50** evaluates overall detection and localization performance
- **Precision** ensures reliability by reducing false detections
- **Recall** ensures robustness by detecting as many UAVs as possible

1.3 Tools

MLOps stage	Tools	Purpose
Data Augmentation and Labeling	Roboflow	Perform image annotation and apply data augmentation to improve UAV detection robustness and generalization
Data Versioning	Hugging Face + Git	Manage dataset versions, track changes, and ensure reproducible training workflows

Experiment Tracking and Model Registry	W&B	Log training experiments, compare model performance metrics, and store best-performing UAV detection models
Pipeline Orchestration and Workflow Automation	ZenML	Automate and orchestrate end-to-end ML pipelines in a reproducible and tool-agnostic manner
Model Deployment and serving	NixOS + ONNX	Optimize and deploy the trained UAV detection model for offline, efficient, and reproducible inference on edge or mobile platforms
Monitoring, Drift Detection and Governance	Evidently AI	Monitor data drift, detect performance degradation, and ensure long-term reliability of the deployed model

1.4 Environmental Setup

The project environment is designed to ensure reliable deployment in offline under resource-constrained settings. Development is performed using **Python 3.10** within an isolated virtual environment, with dependencies centrally managed to maintain a consistent and reproducible runtime.

Dataset annotation and augmentation are carried out using **Roboflow** to enhance model generalization. Dataset versioning is managed through **Hugging Face repositories integrated with Git**, enabling controlled tracking of data changes without dependence on cloud-only infrastructure.

Weights & Biases (W&B) is used for experiment tracking and model management, logging training configurations, metrics, and artifacts for systematic comparison of YOLO-based models. The end-to-end machine learning workflow, including training and evaluation, is orchestrated using **ZenML**, providing a modular and tool-agnostic MLOps pipeline.

For deployment, the trained model is converted to **ONNX** and deployed using **NixOS**, ensuring reproducible and efficient inference on edge or mobile platforms. **Evidently AI** supports performance monitoring and data drift detection, while a lightweight offline dashboard enables real-time visualization in disconnected environments.

1.4 Literature Review

YOLO (You Only Look Once) introduced a unified, single-stage detection framework that treats detection as a regression problem, enabling real-time performance by predicting bounding boxes and class probabilities in a single network pass. This seminal work established the foundation for subsequent YOLO family improvements focused on speed–accuracy tradeoffs.

Subsequent versions—YOLOv3 and YOLOv4—provide incremental architectural and training improvements that increase accuracy while preserving real-time throughput; these variants are commonly chosen for UAV/drones tasks because they balance detection quality and inference latency. (*Practical note:* Ultralytics’ YOLOv5/YOLOv8 implementations are widely used in engineering projects though they are primarily released as code/docs rather than peer-reviewed papers.)

For tracking objects across frames, light-weight online methods are commonly paired with fast detectors:

- SORT (Simple Online and Realtime Tracking) uses a Kalman filter for motion prediction and the Hungarian algorithm for data association; its simplicity yields very high update rates appropriate for real-time pipelines.
- Deep SORT extends SORT by adding a learned appearance descriptor (deep association metric) to reduce identity switches under occlusion—often used alongside YOLO detectors in UAV tracking systems.

These trackers are standard engineering choices when the emphasis is on throughput and robustness for practical UAV surveillance.

Deploying detection models on mobile/edge platforms requires model conversion, hardware-aware optimizations, and lightweight runtimes. The ONNX format and ONNX Runtime (including ONNX Runtime Mobile) are widely used to convert models from PyTorch/TensorFlow and run optimized inference across mobile and IoT platforms. Techniques such as quantization, pruning, and graph optimizations are heavily employed to reduce latency and memory footprint for real-time use.

Survey articles on on-device and edge AI summarize the common optimization pipeline (architecture selection, compression, quantization, hardware acceleration) used to make CV models feasible on constrained devices.

Recent surveys synthesize challenges unique to UAV detection—small object sizes, viewpoint changes, motion blur, and limited labeled multi-class aerial datasets—and discuss solutions including data augmentation, multi-scale feature learning, and fusion with non-visual sensors. These surveys also point to gaps in dataset diversity and stress the importance of end-to-end system evaluation (accuracy + latency) for real deployments

1. Drone Navigation Using Region and Edge Exploitation-Based Deep CNN, Muhammad Arif Arshad et al., IEEE Access

This paper focuses on UAV navigation rather than direct detection, proposing a deep CNN that leverages both region-based and edge-based visual features. The model enables drones to better understand environmental structure and obstacles for autonomous navigation. The study demonstrates how CNN-based perception systems can enhance UAV situational awareness in real-world environments.

2. YOLBO: You Only Look Back Once – A Low Latency Object Tracker Based on YOLO and Optical Flow, Daniel S. Kaputa, Brian P. Landy, IEEE Access

YOLBO introduces a low-latency tracking framework that integrates YOLO-based object detection with optical flow for motion estimation. By reusing past detection information, the tracker reduces computational overhead while maintaining real-time performance. This approach is particularly effective for high-frame-rate applications where fast and stable object tracking is required.

3. Drone Navigation Using Region and Edge Exploitation-Based Deep CNN, Muhammad Arif Arshad et al., IEEE Access

This paper focuses on UAV navigation rather than direct detection, proposing a deep CNN that leverages both region-based and edge-based visual features. The model enables drones to better understand environmental structure and obstacles for autonomous navigation. The study demonstrates how CNN-based perception systems can enhance UAV situational awareness in real-world environments.

4. MLOps Architecture for Machine Vision: A Practical Architecture for Industry Application, Matti Linna

This work presents a practical MLOps architecture tailored for machine vision systems in industrial environments. It emphasizes reproducibility, modular pipeline design, and continuous monitoring across data, model, and deployment stages. The paper highlights how structured MLOps workflows reduce technical debt and improve reliability in production-grade computer vision systems, directly supporting the need for lifecycle management in real-time UAV detection applications.

5. Toward the Reliable Deployment of Computer Vision Pipelines by Leveraging MLOps and Explainable AI, Evangelos Nerantzis, Georgios Symeonidis – ICAITA

This paper explores the integration of MLOps and Explainable AI (XAI) to improve the reliability and trustworthiness of computer vision pipelines. The authors demonstrate how automated pipelines, experiment tracking, and monitoring enhance deployment stability while explainability improves system interpretability. The study reinforces the importance of MLOps practices in maintaining consistent performance of vision models in real-world scenarios.

6. A Study on the Efficiency and Cost-Effectiveness of Edge AI Deployment and Monitoring via MLOps Frameworks Compared to Centralized Cloud Models, Suleman Asghar

This study analyzes the trade-offs between edge-based and cloud-based AI deployments using MLOps frameworks. It demonstrates that edge AI systems, when supported by proper MLOps pipelines, achieve lower latency, reduced bandwidth usage, and improved cost efficiency. The findings validate the suitability of offline and edge-based UAV detection systems supported by automated deployment and monitoring mechanisms.

7. Pipeline-Driven Handwritten Digit Recognition: A ZenML Framework Approach for Efficient Machine Learning Workflows, Ramamoorthy A.S et al., IEEE

The authors present a pipeline-driven machine learning workflow using ZenML for handwritten digit recognition. The study showcases how modular pipelines, experiment tracking, and reproducible workflows simplify model development and deployment. Although applied to digit recognition, the methodology directly aligns with UAV detection pipelines by demonstrating the effectiveness of ZenML in managing end-to-end ML lifecycles.

Chapter 2

Understanding MLOps, Risks, And Requirement Analysis

2.1 What is MLOps?

MLOps is a set of practices that combines Machine Learning, DevOps, and Data Engineering to deploy, monitor, and maintain machine learning models in production environments. MLOps bridges the gap between model development (performed by data scientists) and model deployment (handled by engineers and operations teams), ensuring that ML systems are reliable, scalable, and maintainable throughout their lifecycle.

In the context of the Real-Time UAV Detection and Tracking System, MLOps enables us to:

- **Dataset Versioning:** MLOps helps track and version UAV image datasets and bounding box annotations as data changes over time.
- **Model Experimentation:** It enables systematic experimentation with CNN/YOLO models and hyperparameters to improve detection accuracy.
- **Automation:** MLOps automates the training, validation, and deployment pipeline for real-time UAV detection.
- **Monitoring & Retraining:** It continuously monitors model performance and detects data drift, triggering retraining when needed.
- **Reliability & Compliance:** MLOps ensures the UAV detection system remains accurate, robust, and reliable in real-world conditions.

2.2 Core MLOps Principles

1. Data Management & Versioning — Roboflow + Hugging Face

Roboflow is used for UAV image labeling and data augmentation, while Hugging Face enables dataset versioning to track changes in images and annotations over time.

2. Code and Model Version Control — Git + Hugging Face

Git manages source code versions, and Hugging Face Hub stores and versions trained UAV detection models for reproducibility and rollback.

3. Experiment Tracking — Weights & Biases (W&B)

W&B tracks training metrics, hyperparameters, and evaluation results, enabling comparison of different CNN/YOLO experiments.

4. Pipeline Orchestration & Automation — ZenML

ZenML automates the end-to-end ML pipeline, including data ingestion, model training, evaluation, and deployment.

5. Model Deployment & Serving — NixOS + ONNX Runtime

NixOS ensures reproducible deployment environments, while ONNX Runtime enables optimized and fast inference on mobile and edge platforms.

6. Monitoring & Drift Detection — Evidently AI

Evidently AI monitors data and prediction drift caused by changes in environment, lighting, or background after deployment.

7. Continuous Retraining — ZenML + Hugging Face

ZenML retriggers training pipelines using updated datasets from Hugging Face when drift or performance degradation is detected.

8. Reliability & Long-Term Maintenance — Evidently AI + W&B

Evidently AI detects performance degradation, and W&B helps validate improved models before redeployment.

2.3 Requirement Analysis

Functional Requirements

ID	Requirement	Description
FR1	UAV Data Input	The system shall accept UAV images and real-time video streams as input from cameras or mobile platforms.
FR2	Data Preparation & Versioning	The system shall support UAV data annotation, augmentation, and dataset versioning for consistent model training.
FR3	Model Training	The system shall train CNN/YOLO-based models for accurate UAV detection and bounding box localization.
FR4	Experiment Tracking	The system shall track training metrics, hyperparameters, and model versions for reproducibility and comparison.

FR5	Automated MLOps Pipeline	The system shall automate data preprocessing, training, evaluation, and deployment using MLOps pipelines.
FR6	Model Deployment	The system shall deploy optimized models (ONNX format) for real-time UAV detection on mobile or edge devices.
FR7	Monitoring & Drift Detection	The system shall monitor deployed model performance and detect data or prediction drift over time.
FR8	Continuous Retraining	The system shall retrain and redeploy the model when performance degradation or drift is detected.

Non-Functional Requirements

ID	Requirement	Target	Tool
NFR1	Performance	≥ 25 FPS, ≤ 40 ms inference latency	YOLO, ONNX Runtime
NFR2	Scalability	Support ≥ 5 device types with consistent accuracy	ZenML, NixOS
NFR3	Reliability	$\geq 90\%$ detection accuracy, $\leq 10\%$ false positives.	Evidently AI, W&B
NFR4	Reproducibility	100% reproducible builds across retraining runs	Git, Hugging Face, NixOS
NFR5	Maintainability	Model update cycle ≤ 24 hours after drift detection.	ZenML, Evidently AI

2.4 Risk Matrix

Likelihood \ Impact	Minor	Moderate	Severe	Critical
High	False Positives (Birds as UAVs)	Model Drift (Environmental Changes)	Background Bias / Spurious Correlation	Missed UAV Detection (False Negatives)
Medium	Integration Issues (Camera/Input)	Overfitting due to Single-Class Data	Model Generalization Failure	Security Misclassification

Low	CI/CD Pipeline Failure	Deployment Configuration Errors	Scalability Constraints	Data Availability Issues
Very Low	Logging Failures	Monitoring Visualization Errors	Legal / Compliance Risk	Hardware Failure

Table 2.1 : Risk Matrix

The risk matrix evaluates potential threats to the UAV detection and tracking system by considering two dimensions: **likelihood** and **impact**. Likelihood represents the probability of a risk occurring during system operation, while impact reflects the severity of consequences if the risk occurs. This matrix helps prioritize risks that require stronger mitigation strategies, especially in security-critical and real-time edge deployments.

2.5 Risk Analysis

Likelihood Levels

- **High:** Risks that are expected to occur frequently due to environmental variability, dataset limitations, or operational conditions. These risks require immediate attention and robust mitigation.
- **Medium:** Risks that may occur under certain conditions such as deployment changes, dataset bias, or system integration issues. These risks are manageable but must be monitored.
- **Low:** Risks that are unlikely under normal operating conditions but may arise due to configuration or scalability constraints.
- **Very Low:** Rare risks that occur infrequently and typically result from exceptional conditions such as hardware faults or regulatory constraints.

Chapter -3

Data understanding, Feature engineering and governance

3.1 Data Collection and Exploration Techniques

The dataset used in this project consists of UAV images collected for the purpose of real-time drone detection and tracking. The dataset contains approximately 51.4k images with bounding box annotations and is primarily a single-class dataset focused on UAV objects. Data was sourced in a structured format suitable for object detection tasks.

Exploratory data analysis was performed to understand the distribution of image resolutions, bounding box dimensions, and object density per image. Sample images were visualized to inspect annotation quality and detect inconsistencies. This exploration helped identify dataset limitations such as limited environmental diversity and guided the need for data augmentation.

3.2 Data Profiling, Cleaning and Preprocessing

Data profiling involved examining annotation completeness, bounding box validity, and image quality. Images with missing or incorrect labels were removed to ensure training reliability. Preprocessing steps included resizing images to the required input dimensions for the YOLO model and normalizing pixel values.

To improve robustness, data augmentation techniques such as flipping, cropping, shearing, saturation adjustment, exposure variation, and blur were applied. These preprocessing steps reduced overfitting and enhanced generalization without introducing new classes.

3.3 Feature Extraction and Feature Selection

Feature extraction is performed automatically using Convolutional Neural Networks (CNNs) within the YOLOv-12 architecture. CNN layers learn hierarchical spatial features such as edges, textures, and object shapes directly from raw pixel data.

Manual feature selection is not required in this approach, as the network optimizes relevant features during training. The extracted features enable accurate bounding box prediction and object localization for UAV detection.

3.4 Model Design Approaches

The project employs a CNN-based YOLO object detection architecture designed for real-time performance. YOLO processes the entire image in a single forward pass, allowing simultaneous prediction of bounding box coordinates and object confidence scores.

The model is optimized using the AdamW optimizer with carefully selected hyperparameters such as `weight_decay`, `optimizer (adamw)`, `momentum`, `weight_decay`, `warmup_epochs`, `warmup_momentum` and `warmup_bias_lr`. This design ensures a balance between detection accuracy and inference speed, making it suitable for mobile and edge deployment.

3.5 Governance and Data Version Control

Governance and data version control play a critical role in ensuring reproducibility, traceability, and reliability throughout the machine learning lifecycle. In this project, governance practices are applied to manage datasets, experiments, and trained models in a structured and transparent manner.

Dataset versioning is handled using the **Hugging Face Hub**, which provides **Git**-based version control for datasets and models. Each dataset update is tracked through commits, enabling clear visibility of changes over time. This ensures that the exact dataset version used for training can always be retrieved, supporting reproducible experimentation and collaborative development.

Experiment governance is achieved using **Weights & Biases (W&B)**, which records training metrics, hyperparameters, and model artifacts for each training run. This creates a documented history of the model development process and ensures traceability between datasets, training configurations, and resulting models. Model versions are managed using the W&B Model Registry to maintain consistency and reproducibility.

Pipeline-level governance is supported through **ZenML**, which enforces structured, step-based pipelines for data processing, training, and deployment. ZenML ensures that each pipeline execution follows a defined workflow, reducing human error and improving maintainability. Together, these tools establish a robust governance framework that supports reliable MLOps practices for real-time UAV detection.

Chapter 4

Model Building, Training and Evaluation

4.1 Model Building

The model used in this project is a CNN-based object detection model built using the YOLO architecture. YOLO is designed for real-time object detection and is well suited for UAV detection due to its high inference speed and accuracy. The CNN backbone automatically extracts spatial features such as edges, shapes, and textures from UAV images, which are then used to predict bounding box coordinates and object confidence scores.

The model architecture processes the entire image in a single forward pass, enabling efficient detection and localization of UAVs. Hyperparameters such as learning rate, optimizer type, momentum, and weight decay are manually configured prior to training to ensure stable learning and optimal performance.

4.2 Training, Validation and Testing

The dataset is divided into training, validation, and testing subsets to ensure unbiased evaluation of the model. The training set is used to learn model parameters, while the validation set is used to monitor performance during training and tune hyperparameters. The testing set is used only for final evaluation.

During training, data augmentation techniques are applied to improve generalization and reduce overfitting. The model is trained for multiple epochs, and performance is continuously monitored using validation metrics. This separation ensures that the model does not memorize training data and performs reliably on unseen UAV images.

4.3 Evaluation Metrics and Visualization

The performance of the UAV detection model is evaluated using standard object detection metrics and visualized through training and validation graphs. These metrics provide insights into model learning, localization accuracy, and overall detection performance.

The **training loss metrics** include **box loss**, **classification loss**, and **distribution focal loss (DFL)**. The steady decrease in these losses across epochs indicates that the model is learning accurate bounding box localization, class prediction, and distribution refinement. Similarly, the **validation losses** follow a decreasing trend, demonstrating good generalization and absence of significant overfitting.

Precision and recall metrics are used to measure detection reliability and completeness. The increasing precision curve indicates that the model produces fewer false positives over time, while the rising recall curve shows improved detection of UAV instances. Both metrics stabilize after several epochs, suggesting convergence.

The primary evaluation metric used is **mean Average Precision (mAP@50)**, which measures detection accuracy at an IoU threshold of 0.5. The consistent increase in mAP@50 reflects improved overlap between predicted and ground-truth bounding boxes. Additionally, **mAP@50-95** evaluates performance across multiple IoU thresholds, providing a stricter assessment of localization quality.

All metrics are visualized across epochs, enabling continuous monitoring of model performance. These visualizations help identify convergence behavior, training stability, and potential performance bottlenecks, supporting effective evaluation of the UAV detection model.

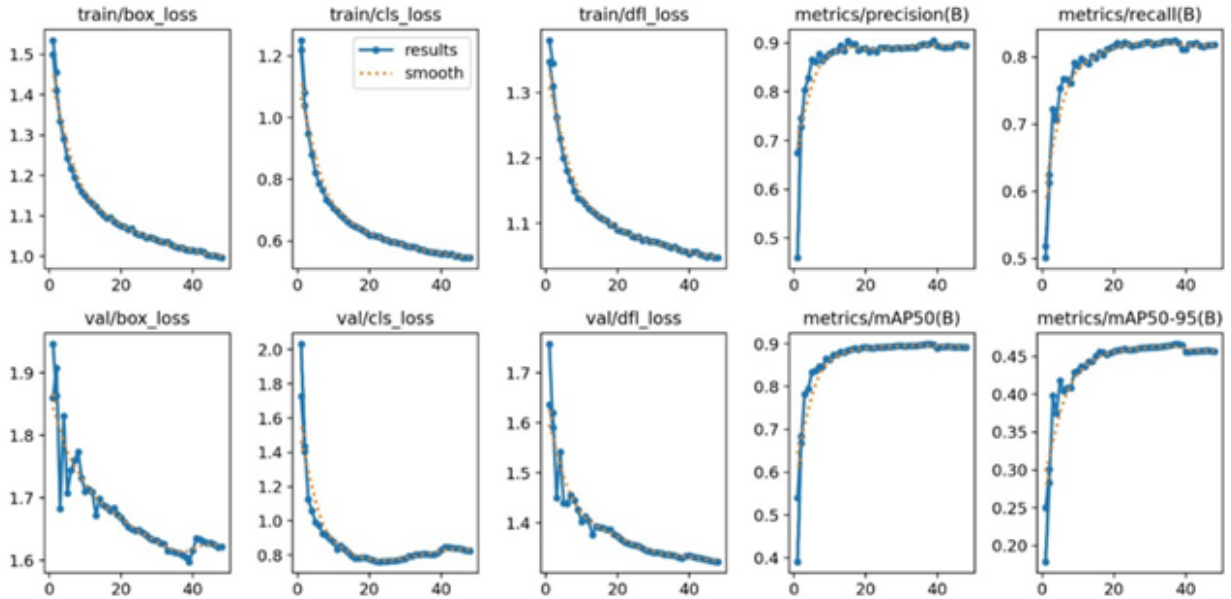


Figure 4.1 Training and validation performance metrics

4.4 Experiment Tracking

Experiment tracking is carried out using **Weights & Biases (W&B)** to record training metrics and hyperparameters. During training, metrics such as loss and mAP@50 are logged

automatically and synced to the W&B cloud. Hyperparameters used for training are stored along with the corresponding metrics to maintain reproducibility.

Although multiple model comparisons are not performed in this project, experiment tracking provides a clear record of the training process and ensures transparency. This helps in monitoring training behavior and maintaining consistent documentation of model development.



Figure 4.2 W&B training metrics dashboard

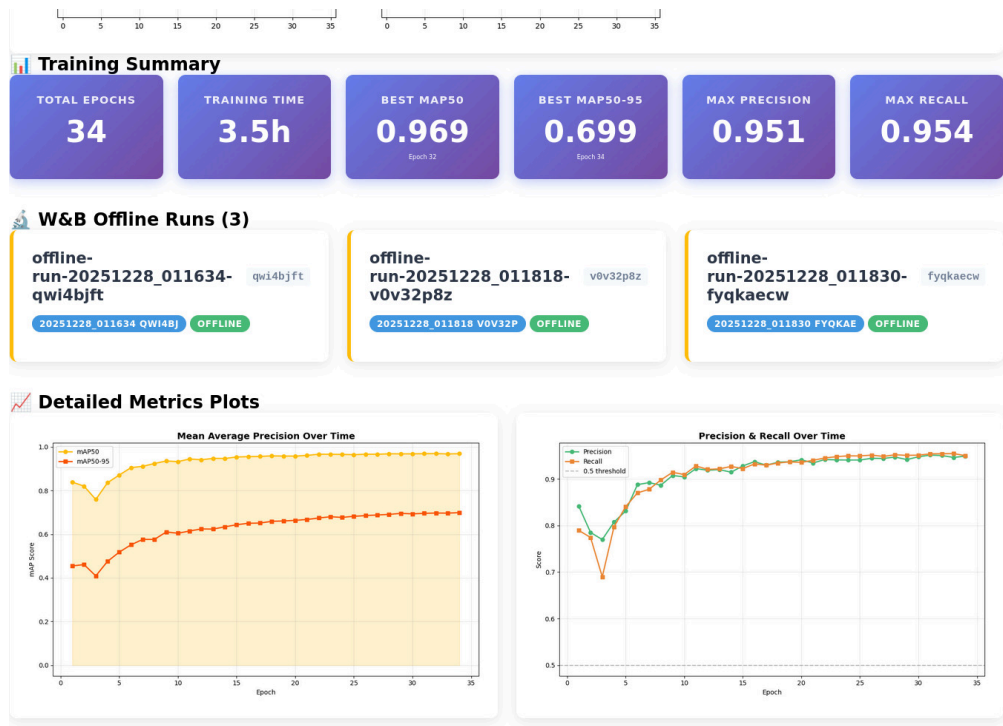


Figure 4.3 W&B training summary

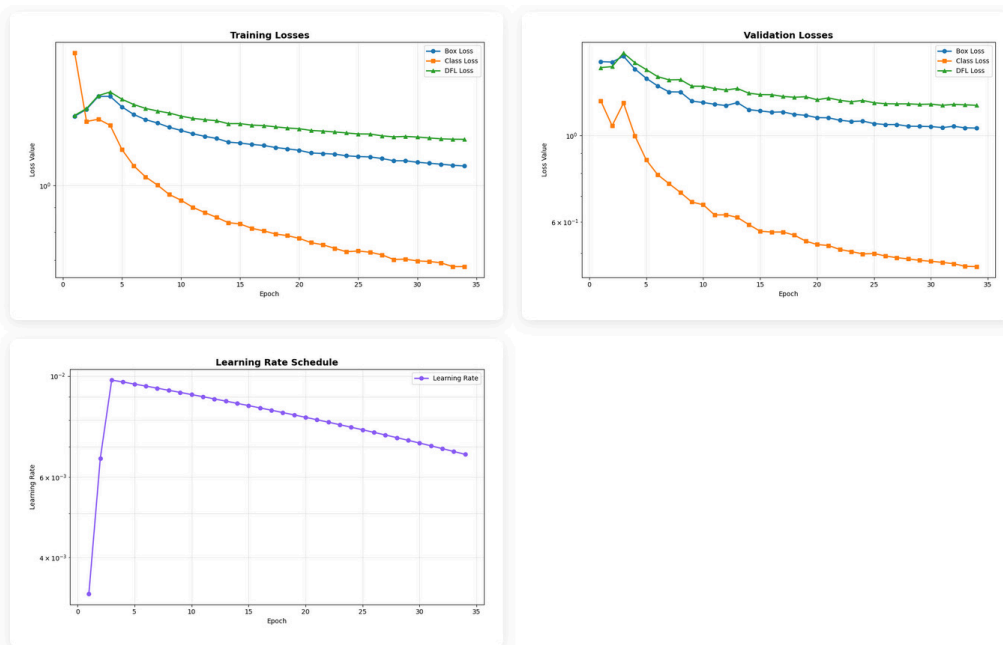


Figure 4.4 Graphs of training losses, validation losses and learning rate schedule

4.5 Interpreting Model Results and Error Analysis

Model results are interpreted by analyzing training and validation curves, precision-recall behavior, and mAP trends. A consistent decrease in loss and an increase in mAP indicate effective learning and convergence. Differences between training and validation performance are used to detect potential overfitting.

Error analysis involves examining incorrect detections such as missed UAVs or inaccurate bounding boxes. These errors are analyzed to identify possible causes such as challenging backgrounds, lighting variations, or motion blur.

Chapter-5

Model Analysis and Refinement

5.1 Revisiting Model Performance

After completion of training, the performance of the YOLO-based UAV detection model was revisited using training, validation, and testing metrics. The model demonstrated consistent learning behavior, as indicated by a steady decrease in training and validation losses across epochs. This suggests that the model effectively learned spatial and contextual features required for UAV detection.

Evaluation metrics such as **precision**, **recall**, and **mean Average Precision (mAP@50)** were used to assess detection quality. The achieved precision indicates that the model produces a low number of false positives, while recall confirms its ability to detect most UAV instances present in the images. The mAP@50 score reflects reliable bounding box localization at an IoU threshold of 0.5, which is suitable for real-time detection scenarios.

The close alignment between training and validation performance indicates minimal overfitting. This validates the effectiveness of data augmentation, regularization, and hyperparameter selection used during training.

5.2 Hyperparameter tuning and optimisation

Hyperparameters play a critical role in determining training stability and final model performance. In this project, key hyperparameters such as **learning rate**, **optimizer (AdamW)**, **momentum**, **weight decay**, and **warmup parameters** were manually configured prior to training.

The AdamW optimizer was selected for its ability to provide stable convergence while handling regularization effectively. Momentum was used to accelerate training and reduce oscillations, while weight decay acted as a regularization mechanism to prevent overfitting by penalizing large weights.

Warmup strategies, including warmup epochs and warmup momentum, helped stabilize early training by gradually increasing learning rates. Proper tuning of these hyperparameters resulted in smoother loss curves, faster convergence, and improved detection accuracy. Although extensive hyperparameter search was not performed, the chosen values ensured balanced performance and reliable convergence.

5.3 Interpretability

Interpretability is essential for understanding and trusting model predictions. In this project, interpretability was achieved through visualization of training metrics and inspection of detection outputs. Training and validation loss curves provided insight into the learning dynamics of the model, while precision, recall, and mAP graphs illustrated improvements in detection quality over time.

Bounding box visualizations on sample images were used to analyze model predictions. These visual outputs helped assess localization accuracy, confidence levels, and detection consistency. Error cases, such as missed detections or inaccurate bounding boxes, were manually examined to identify possible causes such as motion blur, lighting variations, or background complexity.

This interpretability process supports model validation and provides insights for future improvements, including data enhancement and architecture optimization.

5.4 Model versioning and Registry

Model versioning is a critical component of MLOps and ensures reproducibility and traceability. In this project, model versioning is managed using the **Weights & Biases (W&B) Model Registry**. Each trained model is stored as a versioned artifact along with its corresponding training configuration, hyperparameters, and evaluation metrics.

The Model Registry enables clear tracking of model evolution and supports easy rollback to previous versions if required. It also facilitates comparison between different training runs and ensures that only validated models are promoted for deployment. This structured approach reduces deployment risk and supports maintainable model lifecycle management.

5.5 Documentation of Model updates

Comprehensive documentation of model updates is maintained throughout the project lifecycle. Each training run, including changes in dataset versions, augmentation strategies, and hyperparameter configurations, is logged using Weights & Biases.

These logs provide a detailed historical record of model development and experimentation. This documentation ensures transparency, supports collaborative development, and simplifies future maintenance or enhancement of the model. By maintaining well-documented model updates, the project adheres to best practices in MLOps and ensures long-term sustainability.

Chapter-6

Model Analysis and refinement

6.1 Overview of Deployment Workflow for Offline Edge ML

In offline edge-based machine learning systems, the deployment workflow focuses on **reproducibility, portability, and stability**, rather than continuous cloud-style deployment. Unlike traditional CI/CD pipelines, this UAV detection system is deployed in environments with limited or no internet connectivity and constrained computation. Therefore, updates to the model and pipeline are performed through **controlled offline packaging and redeployment**, ensuring consistent behavior across devices.

In this project, the deployment workflow ensures that:

- Model training and evaluation are reproducible across runs
- Only validated models are exported and used for deployment
- Inference runs reliably on edge/mobile platforms with minimal dependency issues
- Model versions and dataset changes remain traceable for audit and comparison

This approach reduces runtime failures and supports stable real-time UAV detection in disconnected environments.

Note: For continuous integration in offline conditions, the project supports **limited update synchronization using long-range radio communication**, enabling transfer of validated models/logs without relying on internet-based CI/CD.

6.2 Tools Setup for Reproducible Training and Deployment

Instead of cloud-based CI/CD tools, the project uses lightweight tools that support offline development and reproducibility:

- **Git + Hugging Face repositories** for dataset and version control
- **Weights & Biases (W&B)** for experiment tracking and comparison of YOLO training runs
- **ZenML** for orchestrating modular training and evaluation workflows
- **ONNX** for model export and hardware-friendly inference
- **NixOS** for maintaining a consistent runtime environment across deployments

6.3 Automating Model Training and Testing (Offline):

To ensure consistency and quality, training and evaluation are automated using a structured pipeline. Whenever dataset versions or training configurations change, the pipeline executes the full workflow including preprocessing, training, and evaluation.

The model is evaluated using object detection metrics:

- mAP@50
- Precision
- Recall

Validation checks are performed to prevent weak models from being selected for deployment. Additional offline testing includes:

- dataset format and label consistency checks
- bounding box correctness verification
- overfitting detection using training vs validation performance comparison

This ensures that the final deployed model is robust and reliable for real-time UAV detection.

Chapter-7

Monitoring and tracking the model lifecycle

7.1. Model Monitoring Concepts and KPIs

Model monitoring is critical to ensure that the UAV detection and tracking system remains accurate and reliable under real-world operating conditions. Unlike traditional static software, computer vision models can degrade over time due to changes in environment such as lighting, weather, camera angles, drone size variation, and background complexity. Since the system is intended for real-time inference on mobile/edge platforms, monitoring must cover both **detection quality** and **runtime performance**.

We focus on two categories of Key Performance Indicators (KPIs):

7.1.1 Model Quality Metrics (Detection Performance):

These metrics evaluate how accurately the YOLO-based model detects UAVs and localizes them using bounding boxes.

- **mAP@50** (Mean Average Precision at IoU=0.5):
Measures the overall detection accuracy by evaluating whether predicted bounding boxes overlap sufficiently with ground truth UAV bounding boxes.
- **Precision**:
Measures how many predicted UAV detections are correct. High precision is important to reduce false alarms such as birds, planes, or background objects being detected as UAVs.
- **Recall**:
Measures how many actual UAVs present in the scene are successfully detected. High recall is critical to reduce missed detections (false negatives), which is a major risk in surveillance systems.
- **Tracking Consistency (ID Stability / Track Continuity)** (*optional KPI*):
Evaluates whether the tracking module maintains consistent tracking of the UAV across consecutive frames without frequent loss or identity switching.

7.1.2. Operational Metrics

Since the model runs in real-time on edge/mobile environments, operational KPIs are essential to ensure usability and responsiveness.

- **Inference Latency:**
Time taken to process a single frame and produce detection outputs. Low latency ensures real-time performance.
- **FPS (Frames Per Second):**
Measures how many frames are processed per second. Higher FPS indicates smoother real-time detection and tracking.
- **Resource Utilization:**
Tracks CPU usage, GPU usage (if available), and memory consumption to ensure the system is feasible on limited hardware.

7.2. Detecting Concept Drift and Data Drift

This occurs when the statistical properties of input images or scene conditions change.

Scenario Examples:

- Change in background (open sky → trees/buildings)
- Weather changes (clear → fog/rain)
- Camera angle changes (front view → top view)
- UAV scale changes (near → far, very small drones)

Detection Method (Conceptual):

Drift can be measured using distribution comparisons on:

- brightness and contrast statistics
- bounding box size distribution
- confidence score distribution

- object count per frame

(In offline deployments, drift monitoring reports are generated locally as logs/reports rather than cloud-based continuous monitoring.)

7.2.2 Concept Drift (Performance Drift)

In UAV detection, concept drift refers to situations where the model's detection behavior changes due to new patterns that were not present during training, causing degradation in prediction quality.

Scenario Examples:

- New UAV shapes or designs not seen during training
- Different camera sensors with new noise characteristics
- UAVs appearing as very small objects (harder to detect)

Detection Method:

Performance drift is observed by tracking:

- drop in **mAP@50**
- decrease in **recall**
- increase in false positives
over time or across test scenarios.

7.3. Model Retraining and Redeployment

To maintain reliability, the system supports controlled model updating rather than continuous learning on edge devices. Since edge environments may not support safe retraining due to limited compute and lack of labeled data, retraining is performed in a controlled development environment and then deployed to the edge system after validation.

Trigger for Model Update:

- Significant performance drop in evaluation metrics (e.g., recall decreases below threshold)
- Drift alerts indicating strong domain shift in deployment conditions

Workflow:

- **Dataset Update:** New UAV images/videos are collected and labeled using Roboflow.
- **Versioning:** Dataset versions are maintained using Hugging Face + Git.
- **Training & Validation:** YOLO training is repeated with augmentation and hyperparameter tuning.
- **Model Selection:** Best model is selected based on mAP@50, Precision, and Recall.
- **Deployment Packaging:** Final model is exported to ONNX and deployed in the reproducible NixOS environment.

7.4. Logging, Alerting, and Dashboard Setup

For observability and reliability, the system maintains structured logs and provides an offline dashboard to visualize detection results and performance trends.

Experiment Tracking (W&B):

Each training run logs:

- hyperparameters (learning rate, batch size, epochs, image size)
- metrics (mAP@50, precision, recall)
- artifacts (trained weights, plots, sample predictions)

Offline Alerts / Indicators:

Alerts are triggered locally when:

- FPS drops below expected threshold (e.g., <30 FPS)
- confidence scores become consistently low
- false positives increase in specific environments

Unified Dashboard (Offline):

A lightweight dashboard is used to:

- visualize bounding boxes and tracking output in real-time
- display key performance metrics (mAP@50, precision, recall)
- provide logs and summaries for monitoring without internet dependency

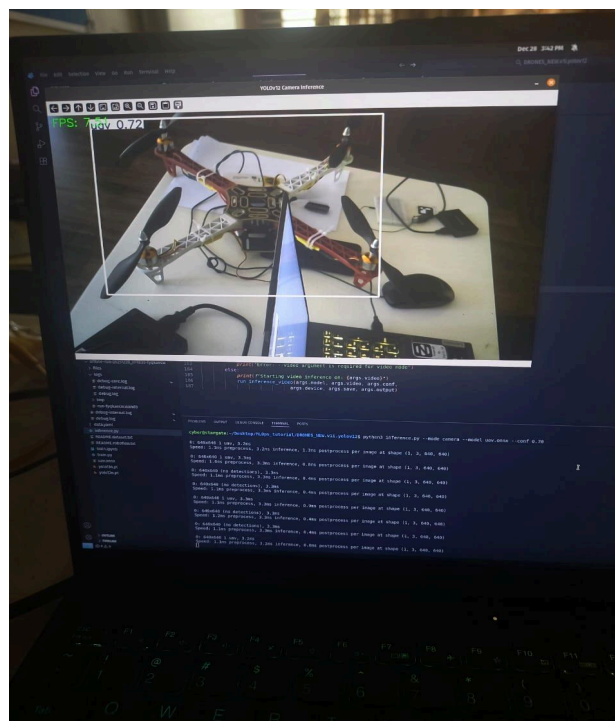


Figure 7.1 Predicted box of UAV

Chapter-8

Performance Evaluation and Governance

8.1. Post-Deployment Performance Review

After deployment, the UAV detection and tracking system undergoes a structured post-deployment performance review using **deployment-like test videos** and **offline inference logs** collected from the edge/mobile environment. The objective of this review is to verify that the deployed YOLO-based model continues to maintain detection accuracy and real-time responsiveness under real-world conditions such as varying backgrounds, lighting, camera angles, and UAV sizes.

All relevant evaluation metrics such as **mAP@50**, **Precision**, **Recall**, and tracking stability indicators are recorded and analyzed using the experiment tracking platform (**W&B**) along with local inference logs. Multiple model versions are compared to identify improvements or regressions across different environmental conditions. Based on these comparisons, the most stable and best-performing model version is selected for continued use in deployment.

This review process ensures:

- Accurate UAV detection with reduced false positives (e.g., birds detected as UAVs)
- Reduced false negatives (missed UAV detections) in surveillance-critical scenarios
- Stable performance across different environments and UAV scales
- Real-time inference feasibility on edge/mobile devices (latency and FPS targets)

8.2. Continuous Improvement via Feedback Loops

To improve long-term reliability, the system supports iterative improvement through controlled feedback loops. Since the deployment environment is offline and does not provide automatic labeling, new training data is collected periodically through offline recordings or curated datasets and then labeled using Roboflow.

The continuous improvement process includes:

- Updating datasets with newly collected UAV images/videos and edge-case samples
- Performing controlled retraining in a development environment using versioned datasets (Hugging Face + Git)
- Re-running the training pipeline and tracking results using W&B

Through this iterative process, the model improves generalization across diverse environments and remains robust to domain shifts such as background changes and varying UAV sizes.

8.3. Governance and Ethical AI Practices

Governance practices are implemented to ensure that the UAV detection system remains reliable, safe, and responsible in real-world deployment. Since UAV detection may be used in surveillance and security applications, incorrect predictions can lead to false alarms or missed threats.

Key governance objectives include:

- Prevention of biased detection due to background correlation (spurious learning)
- Ensuring stable inference behavior under offline constraints
- Maintaining consistent deployment configuration across systems to avoid runtime inconsistencies

Governance is enforced through careful dataset validation, controlled model updates, and performance verification before deployment.

8.4. Audit and Review Checklist

A structured audit and review process is followed to ensure system stability and long-term reliability.

The audit checklist includes:

- Verification of experiment tracking logs and model lineage (W&B runs and artifacts)
- Review of dataset versions and change history (Hugging Face + Git)
- Validation of pipeline reproducibility (ZenML pipeline execution)
- Confirmation of ONNX deployment correctness and inference consistency (NixOS environment)
- Verification of offline dashboard outputs (real-time bounding boxes, tracking, logs)

Regular audits help detect failures early, ensure stable deployment, and maintain reliable UAV detection performance in real-world scenarios.

8.5 Limitations of Monitoring in Offline Edge Systems

Although monitoring is essential for maintaining performance in production ML systems, offline edge deployments introduce practical limitations that restrict full-scale monitoring and continuous retraining. In this project, the UAV detection system is designed for environments with limited internet connectivity and constrained computation, which affects monitoring capabilities.

Key limitations include:

- **Limited access to labeled ground truth:**
Edge deployments typically do not provide immediate ground truth labels, making it difficult to continuously compute mAP@50 or validate detection correctness in real time.
- **Storage constraints for logging:**
Continuous video logging is storage-intensive; therefore, only selected frames, inference summaries, or sampled logs can be retained for monitoring.
- **No safe automatic retraining on edge devices:**
Continuous retraining is avoided since it can introduce background bias and reduce generalization without controlled dataset validation.

Therefore, monitoring in this project is primarily used for **diagnostic analysis and reporting**, while model updates are performed through controlled retraining and validated redeployment.

Appendix

A.1 List Of MLOps Tools, Its Features, Pros And Cons

1. Roboflow

Features:

- Dataset annotation and augmentation
- Dataset versioning
- Export in YOLO-compatible formats

Pros:

- Simplifies data preparation
- Built-in augmentation support

Cons:

- Free tier has usage limits
- Internet dependency

2. Hugging Face Hub

Features:

- Dataset and model versioning
- Git-based tracking of changes
- Easy collaboration and sharing

Pros:

- Strong version control support
- Simple integration with ML frameworks

Cons:

- Large datasets may require storage management

3. Weights & Biases (W&B)

Features:

- Experiment tracking and metric logging
- Hyperparameter logging
- Model versioning and registry
- Interactive dashboards and visualizations

Pros:

- Easy integration with PyTorch and YOLO
- Real-time visualization of training metrics
- Cloud-based access from anywhere

Cons:

- Advanced features may require a paid plan

4. ZenML

Features:

- Pipeline orchestration for ML workflows
- Step-based execution for training, evaluation, and deployment
- Integration with multiple MLOps tools

Pros:

- Enforces structured and reproducible pipelines
- Reduces manual errors in ML workflows

Cons:

- Initial setup may be complex for beginners
- Requires understanding of pipeline concepts

5. Evidently AI

Features:

- Model performance monitoring
- Data drift and concept drift detection
- Data quality analysis
- Interactive dashboards and reports

Pros:

- Easy detection of data and model drift
- Useful for post-deployment monitoring
- Open-source and customizable

Cons:

- Primarily focused on monitoring, not training
- Requires reference and current datasets for comparison

A.2 Tool Installation Guides

1. Using Roboflow

- Roboflow is accessed via a web interface for dataset annotation, augmentation, and export in YOLO format.

2. Using Hugging Face Hub

- `pip install huggingface_hub`
- Authentication using a personal access token enables dataset and model version control.

3. Installing Weights & Biases (W&B)

- `pip install wandb`
- After installation, authenticate using an API key to enable experiment tracking and visualization.

4. Installing ZenML

- pip install zenml
- The zenml init command initializes a structured pipeline environment for MLOps workflows.

5. Installing Evidently AI

- pip install evidently

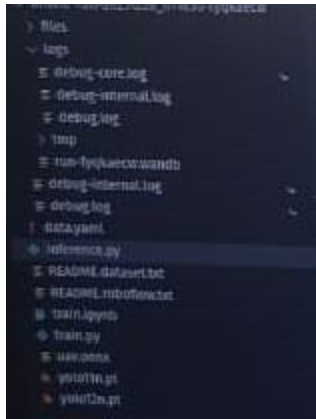
A.3 References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. CVPR.
2. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv.
3. Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv.
4. Du, D., et al. (2018). *The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking*. ECCV (UAVDT).
5. Du, D., et al. (2019). *VisDrone-DET2019: The Vision Meets Drone Object Detection in Image Challenge*. ICCVW.
6. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). *Simple Online and Realtime Tracking (SORT)*. ICIP.
7. Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric (Deep SORT)*. arXiv.
8. ONNX Runtime documentation and mobile/edge deployment guides.
9. Rahman, M. H., et al. (2024). *A Comprehensive Survey of Unmanned Aerial Vehicles Detection and Classification Using Machine Learning*. Remote Sensing.

10. *Heterogeneous Parallelization for Object Detection and Tracking in UAVs*, Mohammed Rabah et al., IEEE Access.
11. *Computationally Enhanced UAV-Based Real-Time Pothole Detection Using YOLOv7-C3ECA-DSA Algorithm*, Siti Fairuz Mat Radzi et al., IEEE Access.
12. *DeTrack: UAV Detection and Tracking Using Neural Networks Ensemble on Surveillance Systems*, Trandafir-Liviu Serghei, Dan Popescu, IEEE Access.
13. *A Dual Mode Detection Method for Unexploded Ordnance Based on YOLOv5 for Low Altitude Unmanned Aerial Vehicle*, Zhongao Ling et al., IEEE Access
14. *Drone Navigation Using Region and Edge Exploitation-Based Deep CNN*, Muhammad Arif Arshad et al., IEEE Access [14]
15. *YOLBO: You Only Look Back Once – A Low Latency Object Tracker Based on YOLO and Optical Flow*, Daniel S. Kaputa, Brian P. Landy, IEEE Access
16. *MLOps Architecture for Machine Vision: A Practical Architecture for Industry Application*, Matti Linna
17. *Toward the Reliable Deployment of Computer Vision Pipelines by Leveraging MLOps and Explainable AI*, Evangelos Nerantzis, Georgios Symeonidis – ICAITA
18. *A Study on the Efficiency and Cost-Effectiveness of Edge AI Deployment and Monitoring via MLOps Frameworks Compared to Centralized Cloud Models*, Suleman Asghar
19. *Pipeline-Driven Handwritten Digit Recognition: A ZenML Framework Approach for Efficient Machine Learning Workflows*, Ramamoorthy A.S et al., IEEE

20. *Real-Time UAV Detection Using an Enhanced YOLOv8 Model*, Ahmed F. Serageldin et al., IEEE [20]

A.4 Folder Structure



A.5 Further Reading

1. Roboflow - <https://roboflow.com/>
2. Hugging Face Hub - <https://huggingface.co/>
3. Weights & Biases (W&B) - <https://wandb.ai/site/>
4. ZenML - <https://www.zenml.io/>
5. Evidently AI - <https://www.evidentlyai.com/>
6. Dataset Link : <https://huggingface.co/datasets/pathikg/drone-detection-dataset>